Lecture 12. Iteration

For loops, while loops, map functions, failure handling, multiple arguments

Contents

12.1 Iteration **2**

12.2 For loops **2**
12.2.1 Variations 2

12.3 For loops versus functionals **3**

12.4 The map functions **4**
12.4.1 Shortcuts 4
12.4.2 Base R 5

12.5 Dealing with failure **5**

12.6 Mapping over multiple arguments **5**

12.7 Walk, reduce and accumulate **6**

12.8 Selected exercises **6**

12.1 Iteration

Motivation. Iteration repeats a computation over many inputs: loops or functionals.

Less duplication: clearer intent, changes in one place, fewer bugs. **Iteration**: *imperative* for/while loops, or *functional* tools (`purrr`) that hide the boilerplate.

12.2 For loops

Median of each column of

```
df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
```

without copying `median(df$a)` four times:

```
output <- vector("double", ncol(df))   # 1. output
for (i in seq_along(df)) {             # 2. sequence
  output[[i]] <- median(df[[i]])       # 3. body
}
output
```

```
## [1] -0.24576245 -0.28730721 -0.05669771  0.14426335
```

Three components:

1. **Output**: preallocate with `vector(type, length)`; growing with `c()` copies everything each step.
2. **Sequence**: `seq_along(df)`, a safe `1:length()`: empty for length 0, whereas `1:length(y)` is `c(1, 0)`.
3. **Body**: run once per `i`; use `[[ ]]` to show one element is handled.

```
y <- vector("double", 0)
seq_along(y)
```

```
## integer(0)
```

```
1:length(y)
```

```
## [1] 1 0
```

12.2.1 Variations

Modifying an existing object. Rescaling every column of `df` (Lecture 10):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
for (i in seq_along(df)) {
  df[[i]] <- rescale01(df[[i]])
}
```

The output is `df` itself.

Looping patterns. Over elements, `for (x in xs)` (awkward for output); over names, `for (nm in names(xs))` with `x[[nm]]`; over indices, the most general (name and value):

```
results <- vector("list", length(x))
names(results) <- names(x)
```

```
for (i in seq_along(x)) {
  name <- names(x)[[i]]
  value <- x[[i]]
}
```

Unknown output length. Not `c()` (quadratic): store pieces in a list, combine once with `unlist()` (`purrr::flatten_dbl()`), `paste(collapse = "")` or `dplyr::bind_rows()`:

```
means <- c(0, 1, 2)
out <- vector("list", length(means))
for (i in seq_along(means)) {
  n <- sample(100, 1)
  out[[i]] <- rnorm(n, means[[i]])
}
str(out)
```

```
## List of 3
##  $ : num [1:38] 0.255 -0.553 1.405 -0.795 -1.567 ...
##  $ : num [1:17] 0.91 2.92 2.3 1.75 1.56 ...
##  $ : num [1:14] 3.215 0.722 2.748 5.392 3.619 ...
```

```
str(unlist(out))
```

```
##  num [1:69] 0.255 -0.553 1.405 -0.795 -1.567 ...
```

Unknown sequence length: `while`. Runs while a condition is `TRUE` (every for loop is a while loop, not conversely). Flips until three heads in a row:

```
flip <- function() sample(c("T", "H"), 1)
flips <- 0
nheads <- 0
while (nheads < 3) {
  if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
  flips <- flips + 1
}
flips
```

```
## [1] 18
```

How many flips on average? [Supplement]

e_k: expected further flips at run length k. $e_3 = 0$, $e_k = 1 + \frac{1}{2}e_{k+1} + \frac{1}{2}e_0$ $(k = 0, 1, 2)$, so $e_0 = 14$; simulation:

```
flips_until_3 <- function() {
  flips <- 0; nheads <- 0
  while (nheads < 3) {
    if (flip() == "H") nheads <- nheads + 1 else nheads <- 0
    flips <- flips + 1
  }
  flips
}
mean(replicate(5000, flips_until_3()))
```

```
## [1] 13.9762
```

12.3 For loops versus functionals

Motivation. Pass the function as an argument: the idea behind purrr.

Loops for means, medians, sds differ only in the function, so make it an argument:

```
col_summary <- function(df, fun) {
  out <- vector("double", length(df))
  for (i in seq_along(df)) {
    out[i] <- fun(df[[i]])
  }
  out
}
col_summary(df, median)
```

```
## [1] 0.6111823 0.4011430 0.4280338 0.5813702
```

```
col_summary(df, sd)
```

```
## [1] 0.3117016 0.2993812 0.3078818 0.2728141
```

A **functional** takes a function as input: `apply()`, `lapply()`, `tapply()`; `purrr` is a consistent family.

12.4 The map functions

One per output type:

function	output
`map()`	list
`map_lgl()`	logical vector
`map_int()`	integer vector
`map_dbl()`	double vector
`map_chr()`	character vector

```
df %>% map_dbl(mean)
```

```
##         a         b         c         d
## 0.5566313 0.4287933 0.4083740 0.5836067
```

```
df %>% map_dbl(sd)
```

```
##         a         b         c         d
## 0.3117016 0.2993812 0.3078818 0.2728141
```

Versus `col_summary()`: written in C, extra arguments via `...` (`map_dbl(df, mean, trim = 0.5)`), names kept. The gain is clarity, not speed (preallocated loops are not slow).

12.4.1 Shortcuts

A model per cylinder group: `split()` into a list and map; in a one-sided formula `.` (`.x`) is the element:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(~lm(mpg ~ wt, data = .))
models %>%
  map(summary) %>%
  map_dbl(~.x$r.squared)
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

A string extracts a named component, an integer a position:

```
models %>% map(summary) %>% map_dbl("r.squared")
```

```
##         4         6         8
## 0.5086326 0.4645102 0.4229655
```

```
x <- list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9))
x %>% map_dbl(2)
```

```
## [1] 2 5 8
```

12.4.2 Base R

`lapply()` ≈ `map()`. `sapply()` simplifies automatically, so its output type depends on the data:

```
threshold <- function(x, cutoff = 0.8) x[x > cutoff]
x1 <- list(c(0.27, 0.37, 0.57, 0.91, 0.20), c(0.90, 0.94, 0.66, 0.63, 0.06),
           c(0.21, 0.18, 0.69, 0.38, 0.77))
x2 <- list(c(0.50, 0.72, 0.99, 0.38, 0.78), c(0.93, 0.21, 0.65, 0.13, 0.27),
           c(0.39, 0.01, 0.38, 0.87, 0.34))
x1 %>% sapply(threshold) %>% str()
```

```
## List of 3
##  $ : num 0.91
##  $ : num [1:2] 0.9 0.94
##  $ : num(0)
```

```
x2 %>% sapply(threshold) %>% str()
```

```
##  num [1:3] 0.99 0.93 0.87
```

A list, then a numeric vector. Safe: `vapply(df, is.numeric, logical(1))` fixes the type.

12.5 Dealing with failure

Motivation. Adverbs turn errors into data, so one failure does not lose all results.

`safely()` never errors; it returns `result` and `error`, one `NULL`:

```
safe_log <- safely(log)
str(safe_log(10))
```

```
## List of 2
##  $ result: num 2.3
##  $ error : NULL
```

```
x <- list(1, 10, "a")
y <- x %>% map(safely(log)) %>% transpose()
is_ok <- y$error %>% map_lgl(is_null)
x[!is_ok]
```

```
## [[1]]
## [1] "a"
```

```
y$result[is_ok] %>% flatten_dbl()
```

```
## [1] 0.000000 2.302585
```

`transpose()` turns a list of pairs into a pair of lists. `quietly()` captures output, messages, warnings; `possibly()` gives a default: `x %>% map_dbl(possibly(log, NA_real_))`.

12.6 Mapping over multiple arguments

`map(mu, rnorm, n = 5)` varies one argument; `map2()` two in parallel; `pmap()` a (named) list of any number. A tibble of parameters drives `pmap()` directly:

```
mu <- list(5, 10, -3)
```

```
sigma <- list(1, 5, 10)
map2(mu, sigma, rnorm, n = 5) %>% str()
```

```
## List of 3
##  $ : num [1:5] 5.45 4.52 4.16 5.48 2.03
##  $ : num [1:5] 17.59 1.02 10.05 8.09 6.07
##  $ : num [1:5] -6.78 7.76 14.72 -6.58 -11.58
```

```
params <- tribble(
  ~mean, ~sd, ~n,
      5,   1,  1,
     10,   5,  3,
     -3,  10,  5
)
params %>% pmap(rnorm) %>% str()
```

```
## List of 3
##  $ : num 3.99
##  $ : num [1:3] 13.2 14.4 12.6
##  $ : num [1:5] -8.43 4.12 -12.89 5.54 -15.36
```

`map2()` is a loop calling `f(x[[i]], y[[i]], ...)`. Varying the function too: `map2(c("runif", "rnorm", "rpois"), param, function(fn, args) exec(fn, !!!args))` (`invoke_map()` is deprecated).

12.7 Walk, reduce and accumulate

walk(), `walk2()`, `pwalk()`: for side effects, returning the input invisibly; `pwalk(list(paths, plot = plots), ggsave, path = tempdir())` saves plots.

reduce() applies a two-argument function repeatedly: `dfs %>% reduce(full_join)`, `vs %>% reduce(intersect)`. **accumulate()** keeps intermediate results:

```
x <- sample(10)
x
```

```
##  [1]  1  7  8  2  6 10  3  9  4  5
```

```
x %>% accumulate(`+`)
```

```
##  [1]  1  8 16 18 24 34 37 46 50 55
```

12.8 Selected exercises

Motivation. Cost of growing, loops over names, the coin-flip mean, map over a vector.

Exercise 12.1 (the cost of growing a vector; after R4DS Exercise 21.2.1.4)

A loop builds a vector of length n either by `out <- c(out, new)` at every step or by filling a preallocated `vector("double", n)`. Assume that `c()` copies all elements of its arguments into a new vector. Count the element copies of each method, and compare them for $n = 10^5$.

Solution [Supplement].

Step 1. At step i, `c(out, new)` writes $i - 1$ old elements and one new: i copies.

Step 2. Total $\sum_{i=1}^{n} i = n(n+1)/2$, quadratic; preallocation: n.

Step 3. Ratio $(n+1)/2$: for $n = 10^5$, about 5×10^9 copies against 10^5. ■

Exercise 12.2 (R4DS Exercise 21.3.5.2)

What happens if you use `for (nm in names(x))` and `x` has no names? What if only some of the elements are named? What if the names are not unique?

```
x <- 1:3
for (nm in names(x)) print(nm)
x <- c(a = 1, 2, c = 3)
names(x)
```

```
## [1] "a" ""  "c"
```

```
for (nm in names(x)) print(x[[nm]])
```

```
## [1] 1
```

```
## Error in x[[nm]]: subscript out of bounds
```

```
x <- c(a = 1, a = 2)
for (nm in names(x)) print(x[[nm]])
```

```
## [1] 1
## [1] 1
```

Solution.

Step 1. No names: `names(x)` is `NULL` (length 0); the loop silently does nothing.

Step 2. Partly named: the others have name `""`; `x[[""]]` matches nothing, so after the first element the loop stops with an out-of-bounds error.

Step 3. Duplicated names: `x[[nm]]` returns the first match, so it is processed twice and the second never.

Step 4. Looping over `seq_along(x)` with `names(x)[[i]]` avoids all three. ■

Exercise 12.3 (flips until k heads in a row)

Generalise the note on the coin-flip loop: let N be the number of fair-coin flips until k heads in a row first occur. Show that $\mathbb{E}N < \infty$ and $\mathbb{E}N = 2^{k+1} - 2$ (which is 14 for $k = 3$).

Solution [Supplement].

Step 1. ($\mathbb{E}N < \infty$.) Blocks of k flips are all heads independently w.p. 2^{-k}, and $N \le mk$ if one of the first m is. So $P(N > mk) \le (1-2^{-k})^m$ and $\mathbb{E}N = \sum_{t\ge 0} P(N > t) \le k \sum_{m\ge 0}(1-2^{-k})^m = k2^k < \infty$.

Step 2. (Equations.) e_j: expected further flips at run length j, $e_k = 0$; one flip leads to $j+1$ or 0 w.p. $\frac{1}{2}$ each:

$$e_j = 1 + \tfrac{1}{2}e_{j+1} + \tfrac{1}{2}e_0, \qquad j = 0, \dots, k-1.$$

Step 3. (Solution.) Try $e_j = 2^{k+1} - 2^{j+1}$. Then $e_k = 0$, and

$$1 + \tfrac{1}{2}(2^{k+1} - 2^{j+2}) + \tfrac{1}{2}(2^{k+1} - 2) = 2^{k+1} - 2^{j+1} = e_j.$$

Step 4. (Uniqueness.) Backwards from $j = k-1$, $e_j = a_j + b_j e_0$ with $b_{k-1} = \frac{1}{2}$, $b_j = \frac{1}{2}b_{j+1} + \frac{1}{2}$, so $b_j = 1 - 2^{-(k-j)}$; $e_0 = a_0 + (1 - 2^{-k})e_0$ fixes the finite e_0. Hence $\mathbb{E}N = e_0 = 2^{k+1} - 2$. ■

Exercise 12.4 (R4DS Exercise 21.5.3.4)

What does `map(-2:2, rnorm, n = 5)` do, and why? What does `map_dbl(-2:2, rnorm, n = 5)` do, and why?

```r
set.seed(1)
map(-2:2, rnorm, n = 5) %>% str()
```

```
## List of 5
##  $ : num [1:5] -2.626 -1.816 -2.836 -0.405 -1.67
##  $ : num [1:5] -1.82 -0.513 -0.262 -0.424 -1.305
##  $ : num [1:5] 1.512 0.39 -0.621 -2.215 1.125
##  $ : num [1:5] 0.955 0.984 1.944 1.821 1.594
##  $ : num [1:5] 2.919 2.7821 2.0746 0.0106 2.6198
```

```r
map_dbl(-2:2, rnorm, n = 5)
```

```
## Error in `map_dbl()`:
## i In index: 1.
## Caused by error:
## ! Result must be length 1, not 5.
```

Solution.

Step 1. `map()` calls `rnorm(.x, n = 5)`; with `n` named, `.x` goes to `mean`: five vectors of five draws, means $-2, \ldots, 2$.

Step 2. `map_dbl()` needs one number per element but gets five: error. Use `map()` and combine, or `map_dbl()` with scalar functions. ■